A.U.R.A. — Full Build Roadmap (FYP-1 + FYP-2)

Scope split (as decided):

- **FYP-1 (Fall semester) — SOFTWARE ONLY.** No UI polish, no website, no physical hardware. Pure working core: model, orchestration, agents, memory, security, execution.
- **FYP-2 (Spring semester) — UI + Website + Hardware.** Desktop UI, marketing/docs website, Raspberry Pi + fingerprint + mic + projector integration, replacing the software-simulated approval built in FYP-1.

Locked technology decisions going into this roadmap (do not re-litigate these mid-build — if something genuinely fails in testing, come back and revise deliberately, not on a whim):

- Single shared local reasoning model: **Qwen3.5-4B**, invoked with different prompts/tools/permissions per logical agent. No BitNet.
- **Deterministic Control Plane, not the LLM, is the "Brain."** The LLM plans and proposes tool calls; a separate Python orchestrator owns state, permissions, routing, risk policy, and execution control.
- OS interaction: accessibility-first (pywinauto/PyAutoGUI), vision only as fallback.
- Memory: ChromaDB (vector) + SQLite (state/log), AES-256 encrypted at rest.
- Dev hardware: 6GB VRAM laptop = target/minimum spec to validate against continuously; 12GB RTX 3060 (brother's PC) = heavy-lifting dev/benchmark machine.
- Real fingerprint/Raspberry Pi hardware does **not** exist yet in FYP-1 — every high-risk action must still go through an approval **interface**, just backed by a software mock until FYP-2 swaps in the real hardware call. Build this boundary cleanly now so FYP-2 is a swap, not a rewrite.

How to use this document: Each stage has an **Entry State** (what must already be true/working before you start), a numbered **Milestone list** (do them in order — each depends on the one before), and an **Exit Criteria** (the exact, testable definition of "this stage is done, next stage can start"). Treat each stage as if a different person is picking it up cold — the Entry State and Exit Criteria are the handoff contract between them, even if it's really just you (or a new Claude session) continuing the work.

STAGE 1 — Foundation & Core Reasoning Engine

Goal: Before building anything else, prove — with numbers, not assumptions — that the chosen model can actually think and plan the way A.U.R.A. needs it to. Nothing gets built on top of an unvalidated brain.

Entry State: Nothing exists yet. This is day one.

Milestones

1.1 — Dev environment setup

- Install Python (3.11+), git, and a virtual environment on both machines (6GB laptop, 12GB dev PC).
- Install Ollama (or llama.cpp directly) on both machines.
- Install core Python libraries you'll need repeatedly: `pywinauto`, `pyautogui`, `chromadb`, `sqlite3` (stdlib), `cryptography`, `pydantic` (for structured tool-call schemas), `requests`.
- Set up a git repository for AURA from day one — commit early, commit often. This is also your "prior art" timestamp record.

1.2 — Model acquisition and raw validation

- Pull Qwen3.5-4B (Q4_K_M quantization) via Ollama on both machines.
- Confirm it loads and responds to a basic prompt on the 6GB laptop specifically — this is your hard floor, not the 12GB machine.
- Record raw numbers on both machines: tokens/sec, first-token latency, VRAM used at idle and under a long-context prompt.

1.3 — Define the tool-call schema

- Design the exact JSON structure the model must output when it wants to call a tool (tool name, parameters, reasoning/justification field, risk hint field).
- Use `pydantic` to define this schema in code so it can be validated automatically — any model output that doesn't match this schema is a failure, not something you manually forgive.
- Write 5-10 example tool-call JSON outputs by hand first, so you know exactly what "correct" looks like before you ask the model to produce it.

1.4 — Build the minimal prompt scaffold

- Write the base system prompt: who A.U.R.A. is, what tools currently exist (start with a fake/placeholder tool list), the required output format from 1.3, and explicit constraints ("you may only call listed tools", "you must justify high-risk actions").
- This prompt will grow over the next stages — treat it as a living file, not a one-time write.

1.5 — Build the A.U.R.A.-specific model benchmark lab This is the most important milestone in Stage 1. Build a small test harness (a Python script that loops through test prompts and scores the output) measuring:

1. Intent accuracy (did it understand what the user actually wanted?)
2. Task-planning accuracy (is the multi-step plan sensible?)
3. Tool-call accuracy (does the JSON match schema, correct tool, correct parameters?)
4. Failure rate (how often does it hallucinate a tool that doesn't exist, or produce invalid JSON?)
5. Tokens/sec (from 1.2, but now under realistic agent prompts, not toy prompts)
6. First-token latency
7. VRAM usage under realistic prompts
8. RAM usage
9. CPU usage
10. Long-context behavior (does quality degrade as conversation history grows?)

- Write at minimum 20-30 realistic test prompts covering simple, medium, and complex multi-step AURA-style commands (file tasks, browser tasks, ambiguous tasks, tasks that should be refused).
- Run this benchmark on **both** machines and record results side by side.

1.6 — Go/no-go decision on the model

- Review the Stage 1.5 numbers honestly. If Qwen3.5-4B's failure rate or planning accuracy is unacceptably poor on realistic prompts, this is the moment to reconsider (not three stages from now). If it's solid, lock it and move on without re-testing every week.

Exit Criteria

- A documented, numbers-backed model choice (not a hunch).
- A finalized tool-call JSON schema with a working validator.
- A working "user text prompt → model produces a schema-valid plan" pipeline — **no real OS execution yet.** Nothing has touched a file, a window, or the internet.
- A benchmark report (even a simple markdown/spreadsheet file) saved in the repo that the next stage — or a defense panel — can be shown.

STAGE 2 — Deterministic Control Plane & Real OS Execution

Goal: Give the plans from Stage 1 hands. Build the non-LLM software skeleton that actually controls risk, permissions, and execution — and prove the File/OS agent can do real work end-to-end.

Entry State: Stage 1's model + schema + prompt scaffold exist and are benchmarked. No execution capability exists yet.

Milestones

2.1 — Build the Control Plane skeleton

- Task state manager: tracks what task is in progress, what step it's on.
- Agent registry: a simple config/dict mapping logical agent name → allowed tools → permission scope.
- This is plain Python, not an LLM. Keep it that way deliberately — this is A.U.R.A.'s core security argument.

2.2 — Build the Risk / Policy Engine

- Define a risk classification table: which tool + parameter combinations count as low-risk (auto-execute) vs high-risk (require approval). Start with an explicit hardcoded list (e.g., `delete_file`, any path outside a whitelisted folder, any credential/key access = high-risk).
- This engine sits between "model proposed a tool call" and "tool actually runs" — no exceptions, even during testing.

2.3 — Build the software-simulated HITL approval interface

- Write a single function, e.g. `authorize_high_risk_action(action_details) -> bool`, that FYP-2 will later swap to call the real Raspberry Pi + fingerprint sensor.
- For FYP-1, implement it as a simple local confirmation (terminal y/n prompt, or a basic "type your approval PIN" check) — the point is the **interface boundary**, not the real biometric hardware.
- Every high-risk action must pass through this function. No shortcuts, even for your own testing convenience — build the discipline now.

2.4 — Build the File/OS Agent's tool functions

- Deterministic file operations: read, write, move, delete, create folder, list directory — implemented in plain Python (`os`/`pathlib`), not left to the model to "figure out."
- Accessibility-tree window/control interaction via `pywinauto`: locate a specific application by process, read its UI tree, interact with a specific control without bringing it to the foreground.

2.5 — Wire Stage 1 → Stage 2 together

- Full pipeline: user text → model plan (Stage 1) → tool calls → Control Plane risk check (2.2) → auto-execute (low-risk) or freeze-and-request-approval (2.3, high-risk) → tool executor (2.4) runs it for real.

- Start with dead-simple real commands: "create a folder called Test on the Desktop," "rename file X to Y," "open Notepad and type a sentence."

2.6 — Build the Verification step

- After every executed action, run a read-back check confirming the action actually happened as intended (file exists at new path, window title changed, etc.) before reporting success to the user.
- A "success" message that wasn't verified is a bug, not a shortcut.

Exit Criteria

- A user can type one real, simple command and watch it: get planned by the model → risk-checked → (if needed) approved via the mock approval function → executed for real on the OS → verified → reported back correctly.
- This works reliably for the File/OS agent specifically (other agents come in Stage 3).
- High-risk actions demonstrably freeze and require the mock approval step — this is your core security demo, working even before real hardware exists.

STAGE 3 — Multi-Agent Expansion, Memory, Vision Fallback & Security Hardening

Goal: Go from "one agent works" to "the full system exists" — all five logical agents, persistent memory, resilient UI interaction, and the full zero-trust content pipeline.

Entry State: Stage 2's File/OS agent works end-to-end through the Control Plane. The approval interface and risk engine exist and are proven.

Milestones

3.1 — Expand to the remaining logical agents

- Add Browser, Communication, Research, and Security agents as new prompt/tool/permission profiles feeding into the **same** shared Qwen3.5-4B model — do not load separate model instances per agent.
- Each agent gets its own tool whitelist registered in the Stage 2.1 agent registry.

3.2 — Browser Agent tools

- Implement browser automation (Selenium/Playwright, or accessibility-hook-based control of the browser window) for reading page content and interacting with elements.

3.3 — Communication Agent tools + Platform-Compliant Interaction Layer

- Implement messaging automation (e.g., WhatsApp Web) via accessibility hooks.
- Build the randomized human-like timing, non-linear cursor movement, and per-platform daily rate limits described in the proposal — this is a real, testable feature, not a footnote.

3.4 — Research Agent tools

- Local file/content search across the user's system.
- If web research is included: retrieval must pass through the sanitization pipeline (3.6) before ever reaching the model.

3.5 — Long-term memory (ChromaDB + SQLite)

- Set up ChromaDB for vector-based memory (user habits, preferences, file paths, past task outcomes).
- Set up SQLite for structured state/session logging.
- Wire memory retrieval into the prompt pipeline: before planning, relevant memory is retrieved and injected as context — the model is not expected to "remember" on its own.
- Test: close AURA, reopen it, confirm it recalls something from a previous session.

3.6 — Zero-trust sanitization pipeline

- Any content retrieved from the web or an external document gets HTML/JavaScript stripped before it is ever placed in the model's context window.
- Explicitly test this against a deliberately malicious test page containing an embedded instruction ("ignore previous instructions and...") to confirm the sanitization + instruction/data separation actually holds.

3.7 — Vision fallback — test the main model first

- Before adding a separate vision model, test Qwen3.5-4B's own multimodal capability (if available in the variant you're using) on UI-screenshot-grounding tasks: can it locate a described element in a screenshot well enough to be useful?
- Only if that's insufficient, integrate a small dedicated vision model (e.g., a lightweight VLM) strictly as a fallback path — triggered only when the accessibility tree lookup fails, never as the default interaction method.

3.8 — Encryption

- Wrap ChromaDB and SQLite storage with AES-256 encryption. Confirm the encryption key never leaves the device and isn't hardcoded in a committed file.

3.9 — Security regression pass

- Re-run the Stage 2 high-risk approval flow against every new agent's tools — confirm nothing added in Stage 3 accidentally bypasses the Control Plane or risk engine.

Exit Criteria

- All five logical agents functional through the one shared model, each correctly scoped to its own tools/permissions.
- Memory persists correctly across sessions and materially changes model behavior (not just stored and ignored).
- The sanitization pipeline demonstrably blocks a test prompt-injection attempt.
- Vision fallback triggers only on accessibility-lookup failure and correctly locates elements in your test cases.
- All local data at rest is encrypted.

STAGE 4 — Integration Testing, Benchmarking, Failure-Proofing & FYP-1 Defense Readiness

Goal: Stop building new features. Break what you built on purpose, fix it, document it, and get it demo-ready.

Entry State: All five agents, memory, vision fallback, and security layers exist individually and have passed their own stage's tests.

Milestones

4.1 — Build a cross-agent task-scenario test suite

- Write 15-25 realistic multi-step scenarios that require more than one agent (e.g., "find all PDFs in Downloads, summarize each, save summaries in a new folder, tell me which is most urgent" — File/OS + Research + Communication-style reporting).
- Include scenarios designed to fail gracefully: ambiguous commands, missing files, a high-risk action buried inside an otherwise low-risk request.

4.2 — Run the full 12-metric benchmark from Stage 1.5 again, at system level

- Intent accuracy, task-planning accuracy, tool-call accuracy, failure rate, tok/s, first-token latency, VRAM, RAM, CPU usage, end-to-end task completion rate, prompt-injection resistance, long-context behavior — but now measured against the full pipeline, not just the raw model.

4.3 — Adversarial / red-team pass

- Deliberately try to break the sandbox: malicious web content, conflicting instructions, requests that sound low-risk but resolve to a high-risk action.
- Every successful "break" gets logged as a bug and fixed before moving on — this list becomes valuable defense material ("we found and fixed X").

4.4 — Hardware-tier validation

- Run the full test suite on both the 6GB laptop and the 12GB dev machine. Document any behavior differences (latency, failure rate) honestly — this becomes your real end-user-hardware documentation, not a guess.

4.5 — Audit trail review

- Confirm every high-risk action attempt (approved or denied) is actually present in the log, with enough detail to reconstruct what happened.

4.6 — Bug-fixing pass

- Work through everything surfaced in 4.1–4.5 before declaring the stage done. Don't let this slide into FYP-2.

4.7 — Documentation and defense prep

- Update the SLR and technical documentation to match what was actually built (not what was planned).
- Prepare a live-demo script covering at least: one simple task, one multi-agent task, one high-risk action correctly frozen and approved via the mock interface, and one blocked prompt-injection attempt.
- Write a clean handoff note for FYP-2: exactly which function(s) need to be swapped for real Raspberry Pi/fingerprint hardware, and what UI/website work assumes about the software's current API/interfaces.

Exit Criteria (FYP-1 complete)

- A fully working, benchmarked, documented A.U.R.A. software core — no UI polish, no physical hardware, no website — that reliably executes real multi-step OS tasks across all five agents under a proven security/approval boundary.
- A written, numbers-backed benchmark report.
- A clean, explicit handoff spec ready for FYP-2.

FYP-2 OVERVIEW (Spring semester — lighter detail, to be expanded closer to the time)

Entry State: FYP-1's software core is complete, tested, and documented per Stage 4's exit criteria.

1. **Desktop UI** — build a real interface around the software core (status view, task history, settings), replacing raw script/terminal interaction.
2. **Website** — informational/marketing site (project overview, docs, possibly a demo video).
3. **Raspberry Pi hardware integration** — wake-word mic (INMP441) + AS608 fingerprint sensor + projector output; the Stage 2.3 `authorize_high_risk_action()` function gets its mock implementation swapped for a real WebSocket call to the Pi, which performs the actual biometric check.
4. **Full headless end-to-end demo** — the closed-laptop, voice-command, projector-feedback, fingerprint-approval scenario described in the original proposal, now running on real hardware instead of a software mock.
5. **Final FYP-2 documentation and defense prep** — updated SLR/technical docs reflecting the hardware integration, final demo rehearsal.

One rule for the whole project

Do not skip a stage's Exit Criteria to "save time." Every stage after Stage 1 depends on the previous one being genuinely solid — a shaky Stage 2 will surface as a confusing, hard-to-diagnose bug three stages later instead of a clean, easy-to-fix one now.